

SettleIQ

04 — Implementation Roadmap

Purpose

This document defines the development sequence for SettleIQ. The project should be developed phase by phase, with each phase completed, tested, and verified before moving to the next.

Supporting documents: **01_PRODUCT_SPEC.md** — What we are building; **02_SYSTEM_ARCHITECTURE.md** — How the system is structured; **03_DATA_SPEC.md** — What data the system uses.

This document defines **the order in which the system will be built**.

Development Principles

- **Build the deterministic foundation first.** The system must reconcile financial data before AI is introduced.
- **AI is not the foundation.** AI is added only after deterministic reconciliation and exception detection are working correctly.
- **One phase at a time.** Do not implement future phases prematurely.
- **Keep the application working.** After every phase, the application should remain runnable.
- **Test before moving forward.** A phase is complete only when its functionality works and basic tests pass.
- **Avoid unnecessary complexity.** Do not introduce microservices, Kubernetes, complex ML pipelines, multi-agent architectures, or unnecessary infrastructure unless explicitly required later.
- **Prioritize the MVP.** The first objective is a complete working finance reconciliation system. Advanced features are added only after the MVP is stable.

PHASE 0 — Project Initialization

Objective: Create the development environment and repository foundation.

Tasks:

- Create GitHub repository.
- Create project directory structure.
- Initialize Git.
- Create .gitignore.
- Create .env.example.
- Set up Python virtual environment.
- Set up FastAPI backend.
- Set up React + Vite frontend.
- Configure Tailwind CSS.
- Verify frontend runs.
- Verify backend runs.
- Establish basic frontend → backend communication.

Completion / Key condition: Frontend → React Application → Backend API → FastAPI → Health Check

PHASE 1 — Database & Data Models

Objective: Create the database foundation required by the application.

Tasks:

- Configure SQLite.
- Configure SQLAlchemy.
- Create database connection.
- Create database models.
- Create tables for payments, settlements, fees, reconciliations, exceptions, investigations, reviews, and audit logs.
- Create initial database setup process.
- Test database CRUD operations.

Completion / Key condition: Create / Read / Update / Store / Retrieve the core financial records.

PHASE 2 — Synthetic Data Generator

Objective: Create realistic financial data for development, testing, demonstration, and evaluation.

Tasks:

- Create a Python data-generation script.
- Generate payments.csv, settlements.csv, fees.csv, and ground_truth.csv.
- Generate normal transactions.
- Generate amount mismatches.
- Generate missing settlements.
- Generate duplicate transactions.
- Generate reference mismatches.
- Generate unknown/ambiguous cases.
- Create Development, Evaluation, and Demo datasets.
- Ensure every generated anomaly has a known ground-truth classification.

Completion / Key condition: python scripts/generate_data.py → complete valid dataset

PHASE 3 — Data Ingestion & Validation

Objective: Allow SettleIQ to receive and validate financial datasets.

Tasks:

- Implement CSV upload API.
- Validate required files.
- Validate required columns.
- Validate data types.
- Validate numeric values.
- Detect malformed records.
- Detect duplicate records.
- Handle missing values.
- Store valid records in the database.
- Return validation results to the frontend.

Completion / Key condition: User uploads payments.csv, settlements.csv, and fees.csv and receives a clear validation report.

PHASE 4 — Deterministic Reconciliation Engine

Objective: Build the core financial reconciliation engine.

Tasks:

- Match payments with settlements.
- Compare payment and settlement identifiers.
- Compare amounts.
- Check applicable fees.
- Apply reconciliation rules.
- Identify matched records.
- Identify unmatched records.
- Generate reconciliation results.

Completion / Key condition: Supported exception categories: AMOUNT_MISMATCH, MISSING_SETTLEMENT, DUPLICATE, REFERENCE_MISMATCH, UNKNOWN.

Completion / Key condition: The reconciliation engine must be deterministic. Do not use the LLM here.

Completion / Key condition: Example: Records processed: 200 / Matched: 164 / Exceptions: 36.

PHASE 5 — Exception Management

Objective: Create the exception-management layer on top of reconciliation.

Tasks:

- Create unique exception IDs.
- Store exception records.
- Store exception type.
- Link exceptions to transactions.
- Store relevant financial records.
- Create exception retrieval APIs.
- Create exception list endpoint.
- Create individual exception detail endpoint.

Completion / Key condition: The application can display Exception ID, Transaction ID, Exception Type, Status, and Relevant Records for every detected exception.

PHASE 6 — Evidence Builder

Objective: Create the structured evidence layer that prepares information for AI investigation.

Tasks:

- Retrieve relevant payment.
- Retrieve relevant settlement.
- Retrieve relevant fee.
- Retrieve related records when required.
- Build structured evidence object.
- Validate evidence completeness.

Completion / Key condition: For an exception, the backend can produce a structured evidence package. No LLM is required yet.

PHASE 7 — AI Investigation Service

Objective: Introduce AI only after the deterministic system is stable.

Tasks:

- Select LLM provider.
- Create AI service abstraction.
- Create prompt builder.
- Define structured AI output.
- Send evidence to LLM.
- Parse response.
- Validate AI response.
- Store investigation result.
- Handle invalid responses.
- Handle API failures.

Completion / Key condition: AI output: Root cause / Confidence / Recommended action / Explanation / Evidence IDs.

Completion / Key condition: The AI investigates an already-detected exception. It does not replace the reconciliation engine.

PHASE 8 — Guardrail & Decision Engine

Objective: Prevent unsafe AI recommendations from becoming automatic financial actions.

Tasks:

- Define confidence threshold.
- Validate required evidence.
- Validate known resolution rules.
- Implement auto-resolution conditions.
- Implement human-review conditions.
- Prevent direct AI state modification.
- Store decision outcome.

Completion / Key condition: AI Recommendation → Guardrail Checks → AUTO_RESOLVE or HUMAN_REVIEW

PHASE 9 — Audit Trail

Objective: Make every important system decision traceable.

Tasks:

- Record EXCEPTION_CREATED.
- Record AI_INVESTIGATION_COMPLETED.
- Record AUTO_RESOLVED.
- Record SENT_TO_REVIEW.
- Record HUMAN_APPROVED.
- Record HUMAN_REJECTED.
- Store timestamp, transaction, exception, action, AI recommendation, confidence, final decision, and actor.

Completion / Key condition: A user can inspect an exception's history and understand how the final decision was reached.

PHASE 10 — Frontend MVP

Objective: Create the complete user-facing MVP.

Tasks:

- **Upload Page:** Upload files → Validate → Start Reconciliation.
- **Dashboard:** Records Processed, Matched, Exceptions, Auto-Resolved, Human Review, Match Rate, Auto-Resolution Rate.
- **Reconciliation Results:** Display transaction-level results.
- **Exception Details:** Display exception, financial records, evidence, AI investigation, confidence, recommendation, and resolution.
- **Review Queue:** Allow Approve, Reject, and Keep Unresolved.

Completion / Key condition: A user can complete the entire MVP workflow through the UI without manually interacting with backend APIs.

PHASE 11 — MVP Integration & Testing

Objective: Connect everything into one reliable end-to-end system.

Tasks:

- End-to-end testing.
- API testing.
- Database testing.
- Exception testing.
- AI failure testing.
- UI testing.
- Edge-case testing.
- Fix critical bugs.

Completion / Key condition: Upload → Validate → Reconcile → Detect Exception → Classify Exception → Build Evidence → AI Investigation → Guardrails → Auto-Resolve / Human Review → Audit → Dashboard Metrics

PHASE 12 — Evaluation System

Objective: Measure whether SettleIQ actually works.

Tasks:

- Evaluate reconciliation accuracy.
- Evaluate exception classification accuracy.
- Evaluate auto-resolution accuracy.
- Evaluate auto-resolution coverage.
- Evaluate false auto-resolution rate.
- Compare system output against ground_truth.csv.
- Produce a reproducible evaluation report.

Completion / Key condition: Use actual measured results. Never fabricate metrics.

PHASE 13 — Settlement Q&A;

Objective: Add the main differentiating feature after the MVP is stable.

Tasks:

- Build question understanding.

- Retrieve relevant data.
- Perform backend financial calculations.
- Build evidence package.
- Send evidence to LLM.
- Return answer + evidence.

Completion / Key condition: Example: user asks, “Why is today’s settlement lower than expected?”

Completion / Key condition: Critical financial calculations are performed by the backend. The AI explains and investigates the results.

PHASE 14 — UX & Engineering Polish

Objective: Make the project feel like a polished product rather than a prototype.

Tasks:

- Improve dashboard design.
- Improve loading states.
- Improve error messages.
- Improve empty states.
- Improve exception investigation UI.
- Add evidence visualization.
- Add useful status indicators.
- Improve API error handling.
- Improve logging.
- Clean unused code.
- Remove unnecessary dependencies.

Completion / Key condition: The application is stable, visually coherent, and ready for demonstration.

PHASE 15 — Deployment

Objective: Make SettleIQ accessible through a working deployment.

Tasks:

- Choose deployment platform.
- Deploy backend.
- Deploy frontend.
- Configure environment variables.
- Configure production LLM API key.
- Verify database behavior.
- Test complete deployed workflow.
- Fix deployment-specific issues.

Completion / Key condition: A reviewer can access and use the working application.

PHASE 16 — Razorpay Submission Preparation

Objective: Prepare the project for evaluation.

Tasks:

- Clean GitHub repository.
- Write final README.
- Add architecture diagram.
- Add setup instructions.
- Add screenshots.
- Add evaluation results.
- Document limitations.
- Record demo video.
- Prepare 5-minute pitch.
- Test the complete demo from a clean environment.

Completion / Key condition: Final demo: Problem → Upload Data → Reconciliation → Exception → AI Investigation → Evidence → Guardrail → Resolution → Metrics → Settlement Q&A.;

Feature Priority

Priority	Features
Tier 1 — MUST WORK	Project Setup; Database; Synthetic Data; Data Validation; Reconciliation; Exception Classification; Evidence Builder; AI Investigation; Guardrails; Human Review; Audit Trail; Dashboard; Evaluation
Tier 2 — SHOULD HAVE	Polished UI; Evidence Visualization; Better Exception Prioritization; Robust Error Handling; Improved Evaluation
Tier 3 — STRETCH	Settlement Q&A; Advanced Analytics; Exception Trends; What-if Analysis

If time becomes limited, **Tier 3 features must be sacrificed before compromising Tier 1.**

Development Rule for Coding Agents

- Read the three project documents.
- Read this implementation roadmap.
- Identify the current phase.
- Implement only the required functionality for that phase.
- Test the implementation.
- Report what was changed.
- Report any unresolved issues.
- Do not start future phases unless explicitly instructed.
- Do not rewrite working components unnecessarily.

Definition of Done

Implementation → Basic Tests → Manual Verification → No Critical Errors → Git Commit → Proceed to Next Phase

Final Development Strategy

FOUNDATION → DATA → RECONCILIATION → EXCEPTIONS → EVIDENCE → AI → GUARDRAILS → HUMAN REVIEW → AUDIT → UI → EVALUATION → Q&A; → POLISH → DEPLOYMENT → SUBMISSION

Core principle: Build a reliable deterministic finance system first, then add AI where reasoning actually provides value.

This roadmap is the **development control document** for SettleIQ. The coding agent should use it to understand where the project is going, but receive instructions to implement only one phase at a time.